Greedy Algorithms

> → **Key Idea**: Make locally optimal choice at each step to reach global optimum

> • **When to use**: Activity selection, coin change, jump games, interval scheduling

> ⊕ **Complexity**: Varies by problem, often O(n log n) due to sorting

> ⚠ **Common Pitfall**: Assuming greedy works without proving optimality or missing counterexamples

Overview

Visual Example

Activity Selection Timeline

```
Timeline:  0   1   2   3   4   5   6
          |---|---|---|---|---|---|

Activity A:   [===========]        (Start: 1, End: 4)
Activity B:       [===========]    (Start: 2, End: 5)
Activity C:               [=======](Start: 4, End: 6)

Greedy Choice (Earliest Finish Time):
1. Select A (Ends at 4)
   - B starts at 2 (< 4) -> Conflict! REJECT
   - C starts at 4 (>= 4) -> No Conflict. SELECT

Result: Optimized Schedule = {A, C}
```

> **Note**: Activity A is selected because it finishes earlier. Activity B is rejected because its start time (2) overlaps with A (ends at 4).

Core Implementation

Activity Selection Problem

```java
import java.util.*;

public class ActivitySelection {
```

```
    static class Activity {
        int start, end;

        Activity(int start, int end) {
            this.start = start;
            this.end = end;
        }
    }

    /**
     * Select maximum number of non-overlapping activities
     */
    public static List<Activity> selectActivities(Activity[] activities) {
        // Sort by end time (greedy choice)
        Arrays.sort(activities, (a, b) -> a.end - b.end);

        List<Activity> selected = new ArrayList<>();
        selected.add(activities[0]); // Always select first activity

        int lastEndTime = activities[0].end;

        for (int i = 1; i < activities.length; i++) {
            // If activity starts after previous one ends, select it
            if (activities[i].start >= lastEndTime) {
                selected.add(activities[i]);
                lastEndTime = activities[i].end;
            }
        }

        return selected;
    }
}
```

Fractional Knapsack

```
import java.util.*;

public class FractionalKnapsack {
    static class Item {
        int weight, value;
        double ratio;

        Item(int weight, int value) {
            this.weight = weight;
            this.value = value;
            this.ratio = (double) value / weight;
        }
    }

    /**
     * Fractional knapsack using greedy approach
     */
```

```
    public static double fractionalKnapsack(Item[] items, int capacity) {
        // Sort by value-to-weight ratio (greedy choice)
        Arrays.sort(items, (a, b) -> Double.compare(b.ratio, a.ratio));

        double totalValue = 0.0;
        int currentWeight = 0;

        for (Item item : items) {
            if (currentWeight + item.weight <= capacity) {
                // Take entire item
                currentWeight += item.weight;
                totalValue += item.value;
            } else {
                // Take fraction of item
                int remainingCapacity = capacity - currentWeight;
                totalValue += item.ratio * remainingCapacity;
                break;
            }
        }

        return totalValue;
    }
}
```

Huffman Coding

```
import java.util.*;

public class HuffmanCoding {
    static class Node implements Comparable<Node> {
        char character;
        int frequency;
        Node left, right;

        Node(char character, int frequency) {
            this.character = character;
            this.frequency = frequency;
        }

        Node(int frequency, Node left, Node right) {
            this.frequency = frequency;
            this.left = left;
            this.right = right;
        }

        @Override
        public int compareTo(Node other) {
            return this.frequency - other.frequency;
        }

        boolean isLeaf() {
            return left == null && right == null;
```

```
        }
    }

    /**
     * Build Huffman tree using greedy approach
     */
    public static Node buildHuffmanTree(Map<Character, Integer> frequencies) {
        PriorityQueue<Node> minHeap = new PriorityQueue<>();

        // Create leaf nodes
        for (Map.Entry<Character, Integer> entry : frequencies.entrySet()) {
            minHeap.offer(new Node(entry.getKey(), entry.getValue()));
        }

        // Build tree greedily
        while (minHeap.size() > 1) {
            Node left = minHeap.poll();
            Node right = minHeap.poll();

            Node merged = new Node(left.frequency + right.frequency, left, right);
            minHeap.offer(merged);
        }

        return minHeap.poll();
    }

    /**
     * Generate Huffman codes
     */
    public static Map<Character, String> generateCodes(Node root) {
        Map<Character, String> codes = new HashMap<>();
        generateCodesHelper(root, "", codes);
        return codes;
    }

    private static void generateCodesHelper(Node node, String code, Map<Character, S
        if (node.isLeaf()) {
            codes.put(node.character, code.isEmpty() ? "0" : code);
            return;
        }

        generateCodesHelper(node.left, code + "0", codes);
        generateCodesHelper(node.right, code + "1", codes);
    }
}
```

Key Optimizations

1. Sorting Optimization for Intervals

```java
public class IntervalScheduling {

    /**
     * Optimized interval sorting with custom comparator
     */
    public static int maxNonOverlappingIntervals(int[][] intervals) {
        if (intervals.length == 0) return 0;

        // Sort by end time with tie-breaking by start time
        Arrays.sort(intervals, (a, b) -> {
            if (a[1] != b[1]) return a[1] - b[1]; // End time
            return a[0] - b[0]; // Start time for ties
        });

        int count = 1;
        int lastEnd = intervals[0][1];

        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] >= lastEnd) {
                count++;
                lastEnd = intervals[i][1];
            }
        }

        return count;
    }
}
```

2. Jump Game Optimizations

```java
public class JumpGame {

    /**
     * Jump Game I - Can reach end?
     */
    public static boolean canJump(int[] nums) {
        int maxReach = 0;

        for (int i = 0; i < nums.length; i++) {
            if (i > maxReach) return false; // Can't reach this position

            maxReach = Math.max(maxReach, i + nums[i]);

            if (maxReach >= nums.length - 1) return true;
        }

        return true;
    }

    /**
     * Jump Game II - Minimum jumps to reach end
```

```
     */
    public static int minJumps(int[] nums) {
        if (nums.length <= 1) return 0;

        int jumps = 0;
        int currentEnd = 0;
        int farthest = 0;

        for (int i = 0; i < nums.length - 1; i++) {
            farthest = Math.max(farthest, i + nums[i]);

            if (i == currentEnd) {
                jumps++;
                currentEnd = farthest;

                if (currentEnd >= nums.length - 1) break;
            }
        }

        return jumps;
    }
}
```

Common Problem Patterns

1. Gas Station Problem

Problem: There are n gas stations along a circular route. Determine if you can complete the circuit.

Example:

```
Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]
Output: 3 (start at station 3)
```

```
public int canCompleteCircuit(int[] gas, int[] cost) {
    int totalGas = 0, totalCost = 0;
    int currentGas = 0, startStation = 0;

    for (int i = 0; i < gas.length; i++) {
        totalGas += gas[i];
        totalCost += cost[i];
        currentGas += gas[i] - cost[i];

        // If we can't reach next station, try starting from next station
        if (currentGas < 0) {
            startStation = i + 1;
            currentGas = 0;
```

```
            }
        }

        // Check if total gas is enough
        return totalGas >= totalCost ? startStation : -1;
    }
```

2. Minimum Number of Arrows

Problem: Burst balloons with minimum arrows. Each balloon has start and end coordinates.

Example:

```
Input: points = [[10,16],[2,8],[1,6],[7,12]]
Output: 2 (one arrow at [6,7] area, another at [12,16])
```

```
    public int findMinArrowShots(int[][] points) {
        if (points.length == 0) return 0;

        // Sort by end coordinate
        Arrays.sort(points, (a, b) -> Integer.compare(a[1], b[1]));

        int arrows = 1;
        int arrowPosition = points[0][1];

        for (int i = 1; i < points.length; i++) {
            // If balloon starts after arrow position, need new arrow
            if (points[i][0] > arrowPosition) {
                arrows++;
                arrowPosition = points[i][1];
            }
        }

        return arrows;
    }
```

3. Task Scheduler

Problem: Given tasks and cooldown time n, find minimum time to finish all tasks.

Example:

```
Input: tasks = ["A","A","A","B","B","B"], n = 2
Output: 8
Explanation: A -> B -> idle -> A -> B -> idle -> A -> B
```

```java
public int leastInterval(char[] tasks, int n) {
    // Count task frequencies
    int[] frequencies = new int[26];
    for (char task : tasks) {
        frequencies[task - 'A']++;
    }

    Arrays.sort(frequencies);

    // Most frequent task
    int maxFreq = frequencies[25];

    // Number of idle slots needed for most frequent task
    int idleSlots = (maxFreq - 1) * n;

    // Fill idle slots with other tasks
    for (int i = 24; i >= 0 && idleSlots > 0; i--) {
        idleSlots -= Math.min(frequencies[i], maxFreq - 1);
    }

    idleSlots = Math.max(0, idleSlots);

    return tasks.length + idleSlots;
}
```

4. Candy Distribution

Problem: Children sit in a line, each has a rating. Give candies such that higher-rated children get more than neighbors.

Example:

```
Input: ratings = [1,0,2]
Output: 5
Explanation: Give [2,1,2] candies respectively.
```

```java
public int candy(int[] ratings) {
    int n = ratings.length;
    int[] candies = new int[n];
    Arrays.fill(candies, 1); // Everyone gets at least 1 candy

    // Left to right: ensure right neighbor with higher rating gets more
    for (int i = 1; i < n; i++) {
        if (ratings[i] > ratings[i - 1]) {
            candies[i] = candies[i - 1] + 1;
        }
    }
```

```
        // Right to left: ensure left neighbor with higher rating gets more
        for (int i = n - 2; i >= 0; i--) {
            if (ratings[i] > ratings[i + 1]) {
                candies[i] = Math.max(candies[i], candies[i + 1] + 1);
            }
        }

        int totalCandies = 0;
        for (int candy : candies) {
            totalCandies += candy;
        }

        return totalCandies;
    }
```

5. Remove K Digits

Problem: Remove k digits from number to make it smallest possible.

Example:

```
Input: num = "1432219", k = 3
Output: "1219"
```

```
    public String removeKdigits(String num, int k) {
        Stack<Character> stack = new Stack<>();

        for (char digit : num.toCharArray()) {
            // Remove larger digits from stack
            while (!stack.isEmpty() && k > 0 && stack.peek() > digit) {
                stack.pop();
                k--;
            }
            stack.push(digit);
        }

        // Remove remaining digits from end
        while (k > 0) {
            stack.pop();
            k--;
        }

        // Build result
        StringBuilder result = new StringBuilder();
        while (!stack.isEmpty()) {
            result.append(stack.pop());
        }
        result.reverse();

        // Remove leading zeros
```

```
        while (result.length() > 1 && result.charAt(0) == '0') {
            result.deleteCharAt(0);
        }

        return result.length() == 0 ? "0" : result.toString();
    }
```

Advanced Variations

Minimum Spanning Tree (Kruskal's Algorithm)

```
import java.util.*;

public class KruskalMST {
    static class Edge implements Comparable<Edge> {
        int src, dest, weight;

        Edge(int src, int dest, int weight) {
            this.src = src;
            this.dest = dest;
            this.weight = weight;
        }

        @Override
        public int compareTo(Edge other) {
            return this.weight - other.weight;
        }
    }

    static class UnionFind {
        int[] parent, rank;

        UnionFind(int n) {
            parent = new int[n];
            rank = new int[n];
            for (int i = 0; i < n; i++) {
                parent[i] = i;
            }
        }

        int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }
            return parent[x];
        }

        boolean union(int x, int y) {
            int rootX = find(x);
            int rootY = find(y);
```

```
            if (rootX == rootY) return false;

            if (rank[rootX] < rank[rootY]) {
                parent[rootX] = rootY;
            } else if (rank[rootX] > rank[rootY]) {
                parent[rootY] = rootX;
            } else {
                parent[rootY] = rootX;
                rank[rootX]++;
            }

            return true;
        }
    }

    public static List<Edge> kruskalMST(List<Edge> edges, int vertices) {
        Collections.sort(edges); // Greedy: sort by weight

        UnionFind uf = new UnionFind(vertices);
        List<Edge> mst = new ArrayList<>();

        for (Edge edge : edges) {
            if (uf.union(edge.src, edge.dest)) {
                mst.add(edge);

                if (mst.size() == vertices - 1) break;
            }
        }

        return mst;
    }
}
```

Stock Buy Sell with Transaction Fee

```
public class StockWithFee {

    /**
     * Best Time to Buy and Sell Stock with Transaction Fee
     */
    public int maxProfit(int[] prices, int fee) {
        int buy = -prices[0]; // Max profit when holding stock
        int sell = 0;         // Max profit when not holding stock

        for (int i = 1; i < prices.length; i++) {
            int prevBuy = buy;
            int prevSell = sell;

            // Either keep holding or buy today
            buy = Math.max(prevBuy, prevSell - prices[i]);

            // Either keep not holding or sell today (pay fee)
```

```
            sell = Math.max(prevSell, prevBuy + prices[i] - fee);
        }

        return sell;
    }
}
```

Interview Tips

Pattern Recognition

- **Optimization problems**: Look for greedy choice property
- **Sorting first**: Many greedy algorithms require sorting input
- **Local optimum**: Each step makes locally best choice
- **No backtracking**: Once choice is made, never reconsider

Common Mistakes

- **Assuming greedy works**: Always verify greedy choice property
- **Wrong sorting criteria**: Choose correct parameter to sort by
- **Missing edge cases**: Empty input, single element, all same values
- **Not proving optimality**: Greedy might give suboptimal results

Implementation Variations

```
// Template for interval problems
public static int greedyIntervals(int[][] intervals) {
    if (intervals.length == 0) return 0;

    // Sort by appropriate criteria (end time, start time, etc.)
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]);

    int count = 1;
    int lastEnd = intervals[0][1];

    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] >= lastEnd) {
            count++;
            lastEnd = intervals[i][1];
        }
    }

    return count;
}

// Template for choosing maximum/minimum
```

```java
public static int greedyChoice(int[] nums) {
    Arrays.sort(nums); // Or custom sorting

    int result = 0;
    for (int i = 0; i < nums.length; i++) {
        // Make greedy choice based on problem
        if (/* condition */) {
            result += nums[i];
        }
    }

    return result;
}
```

Time & Space Complexity

Algorithm	Time Complexity	Space Complexity	Notes
Activity Selection	O(n log n)	O(1)	Sorting dominates
Fractional Knapsack	O(n log n)	O(1)	Sort by ratio
Huffman Coding	O(n log n)	O(n)	Priority queue operations
Jump Game	O(n)	O(1)	Single pass
Gas Station	O(n)	O(1)	Single pass
MST (Kruskal)	O(E log E)	O(V)	Sort edges, Union-Find
Task Scheduler	O(n)	O(1)	Counting sort

← Previous: Top-K & Heaps | Back to Main Guide | Next: Dynamic Programming →